# CS780 Capstone Project: OBELIX: the Warehouse Robot

Rishab K S
251110061
rishabks25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

**Abstract**

This report details a hierarchical reinforcement learning approach for OBELIX, a warehouse robot navigating a POMDP to find, push, and unwedge a box. After initial attempts with PPO and Dueling QN for finding and pushing struggled with sensor aliasing, I decoupled navigation from fault recovery. My final architecture uses a Soft Actor-Critic (SAC) agent with Prioritized Experience Replay (PER) and 4-frame stacking for primary navigation. Simultaneously, a Recurrent PPO module with a GRU acts as an autonomous "unwedger," taking control upon collision to reliably extricate the robot without hand-coded heuristics.

## 1 Competition Result

**Codabench Username:** r_251110061
**Final leaderboard rank (Test Phase):** 73
**Leaderboard rank – Level 1 (static):** 87
**Leaderboard rank – Level 2 (blinking):** 73
**Leaderboard rank – Level 3 (moving + blinking):** 62

**Number of valid submissions made:**

| Phase | Number of submissions |
|---|---|
| Development Phase for Difficulty level 1 | 5 |
| Development Phase for Difficulty level 2 | 4 |
| Development Phase for Difficulty level 3 | 2 |
| Final Submission Phase | 4 |

Table 1: Number of valid submissions made in each competition phase

## 2 Introduction and Problem Description

- **Motivation:** Autonomous warehouse navigation requires agents to reliably interact with objects while avoiding catastrophic collision states. The OBELIX environment draws inspiration from the classic 1992 paper [1], illustrating real demonstrations of reinforcement learning on a robot.
- **Core Task:** The robot must autonomously *find* a grey box, *attach* to it, *push* it, and *unwedge* it toward a designated arena boundary.
- **Environment & Difficulty Levels:** The robot possesses a discrete action space of 5 moves (L45, L22, FW, R22, R45). The task scales across three difficulty levels: Level 1 features a static

box, Level 2 introduces a blinking box, and Level 3 combines a moving and blinking target. Wall obstacles further constrict navigable space.

- **POMDP Challenge:** The environment restricts the agent to a localized 18-bit sensory observation (16 sonar bits for near/far, 1 infrared sensor bit, and 1 stuck bit). Since the robot lacks absolute coordinates and only observes immediate sensor readings, the environment is a Partially Observable Markov Decision Process (POMDP).
- **High-Level Goal:** My objective is to maximize the mean cumulative reward across all difficulties over the 2000-step horizon while maintaining strict robustness against permanent collisions in walled environments.

# 3 Approach and Model Evolution

- Initial Attempts/Evolution across phases: Prior to the final hierarchical architecture, I experimented with several single-policy approaches. Crucially, none of these initial attempts utilized frame stacking, which ultimately proved to be their limiting factor due to severe sensor aliasing.
  - **Dueling DQN with PER (No Reward Shaping):** I initially trained a Dueling DQN to separate state-value and advantage estimation. Without dense rewards, the native +2000 terminal reward was too sparse to find. The agent quickly learned that spinning in place was the mathematically safest way to avoid the -200 wall collision penalty.
  - **Plain SAC with PER (No Reward Shaping):** I transitioned to Soft Actor-Critic to leverage its entropy maximization for better map exploration. However, with sparse rewards, the agent still collapsed into the safe local minimum of continuous rotation.
  - **SAC with Reward Shaping:** I introduced the tactical rewards (penalizing consecutive spins and rewarding forward movement). The agent successfully broke the spinning loop and began exploring, but frequently became wedged in corners. Without a dedicated unwedger, it would panic and revert to spinning in place indefinitely.
  - **Forced Exploration (Alpha Decay + $\epsilon$-greedy):** Suspecting the SAC entropy tuning was failing, I disabled auto-tuning in favor of a hard episode-wise $\alpha$ decay, combined with an $\epsilon$-greedy wrapper. This forced the agent to move physically, but the exploration was too chaotic to learn coherent box-pushing trajectories.
  - **Prior Action Distribution Bias:** To forcefully cure the spinning, I constrained the actor with a prior probability distribution of `[0.1, 0.15, 0.5, 0.15, 0.1]`, heavily subsidizing the forward action (`FWD` = 50%). The agent moved forward aggressively but failed to recognize or attach to the box due to the 1-frame perceptual ambiguity.
  - **Recurrent SAC (GRU):** Attempting to solve the POMDP without frame stacking, I integrated a GRU directly into the SAC actor to process single sequential observations. However, training an end-to-end recurrent actor-critic proved highly unstable, and the policy ultimately degraded back to stationary spinning.
  - **Test Phase: Final Strategy:** I implemented a hierarchical, two-module architecture decoupling the primary navigation from fault recovery. The primary policy is driven by a Soft Actor-Critic (SAC) algorithm [2] enhanced with Prioritized Experience Replay (PER), while an independent Recurrent PPO agent [3] handles unwedging.

- Detailed description of final / best model
  - **Algorithm Family:** Hierarchical Soft Actor-Critic (SAC) combined with Prioritized Experience Replay (PER) for primary navigation, and Recurrent PPO for fault recovery.
  - **State representation:** The raw observation consists of 18 bits (sonars, infrared, and a stuck bit). Because the environment is a strict POMDP—especially challenging in Level 2 (blinking) and Level 3 (moving)—a single frame contains no velocity or temporal data. To combat this, I employ *Frame Stacking*. I maintain a sliding deque of the 4 most recent frames, concatenating them into a 72-dimensional state vector for the SAC actor. This allows the network to infer the trajectory of moving boxes and bridge the gaps when the box blinks out of existence.
  - **Primary Navigation (SAC) & Exploration:** When the Stuck Bit (18th bit) is 0, the SAC actor evaluates the 72-dimensional stacked state. The MLP architecture uses three hidden layers with LeakyReLU activations as shown in 1. SAC was chosen specifically for its maximum entropy objective; by actively rewarding the agent for action stochasticity, it naturally explores the vast, featureless open spaces of the arena without collapsing into deterministic, repetitive loops.

Figure 1: Neural network architecture of SAC agent

  - **Recovery Module (Recurrent PPO):** When the robot collides with a wall, the Stuck Bit flips to 1. At this moment, the SAC module is hard-suspended, the frame stack is cleared, and the GRU-based PPO Unwedger takes exclusive control. I chose a Gated Recurrent Unit (GRU) because unwedging often requires multi-step sequential logic (e.g., turn left, then turn left again, then move forward) that a purely reactive MLP cannot learn. The GRU processes the previous action and maintains a hidden state vector to execute these complex escape maneuvers. Once the Stuck Bit clears back to 0, control seamlessly yields back to the SAC agent. The neural architecture is show in Figure 2.

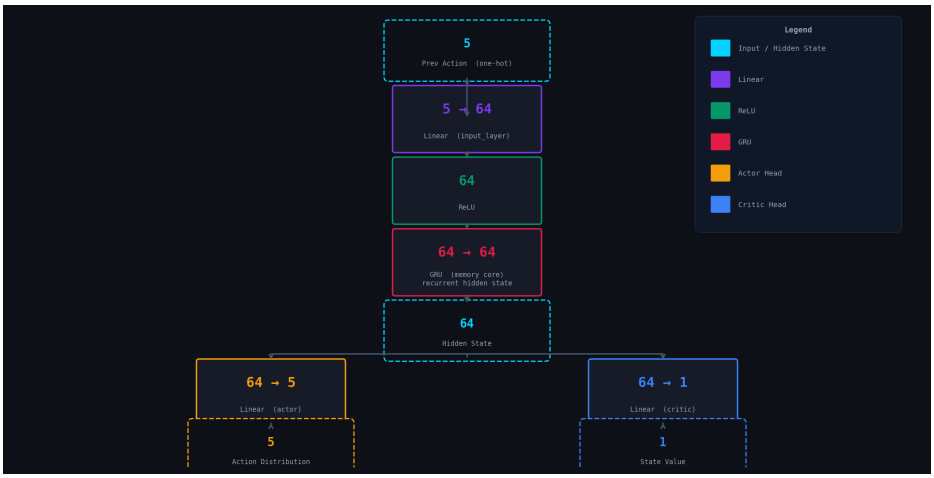



Figure 2: Neural network architecture of Recurrent PPO (Unwedger) agent

- **Reward Shaping:** The native environment rewards are incredibly sparse, making it nearly impossible for the agent to deduce the sequence of finding, attaching, and pushing strictly through random exploration. To solve this, I engineered a multi-tiered reward shaping module:
  - *Survival & Goals:* I established a steep baseline step penalty of $-10.0$ to force urgency. Crashing yields a massive $-500.0$ penalty to instill fear of walls, while attaching to the box grants $+100.0$, and final boundary delivery yields a $+1000.0$ jackpot.
  - *Directional rewards:* I used the immediate 18-bit sensor array to leave tactical breadcrumbs. If the left/right sonars trigger, turning towards the signal is positively reinforced ($+20.0$), while turning away is penalized ($-20.0$). Once the box is locked dead ahead (front sonars or IR active), the `FWD` action is heavily subsidized.
  - *Anti-Stagnation Cruise Control:* A common failure mode was the agent learning that spinning in place was "safer" than exploring. To break this local minimum, I implemented an escalating penalty: driving straight in open space yields a net $-5.0$ reward, but consecutive turning actions accrue an escalating $-10.0 \times \text{spin_steps}$ penalty. This mathematically guarantees that driving forward is the cheapest way to survive in open space.

- **Theoretical Upper-Bound Analysis:** As guided by the problem formulation, I estimated the maximum achievable cumulative return by computing the shortest ideal path between randomized spawn locations and the target box. By mapping the +2000 terminal reward against the -1 step penalties during the push phase, this upper-bound baseline allowed me to diagnose inefficiencies in my SAC agent's initial exploration phase and refine the entropy parameters.

# 4 Experiments and Implementation Details

- **Training setup**: The models were trained for episodes capped at a maximum of 2000 steps. Both the SAC and PPO agent were trained separately for 1000 episodes in the 500x500 arena without walls.

- **Hyperparameters & Optimizers:** The SAC and Recurrent PPO networks were both trained using the Adam optimizer. The specific hyperparameters for these final models are detailed in Table 3 and Table 2, respectively.

| **Hyperparameters** | **Value** |
|---|---|
| *General* | |
| Maximum Training Episodes | 1000 |
| Discount Factor ($\gamma$) | 0.99 |
| *Optimization* | |
| PPO Clipping Parameter ($\epsilon$) | 0.2 |
| Network Update Frequency | 5 |

Table 2: Hyperparameters utilized for the Recurrent PPO (GRU) unwedger module.

| Hyperparameters | Value |
|---|---|
| *Optimization* | |
| Actor Learning Rate | $1 \times 10^{-4}$ |
| Critic Learning Rate | $3 \times 10^{-4}$ |
| Batch Size | 256 |
| Soft Target Update Rate ($\tau$) | 0.01 |
| Network Update Frequency | 5 |
| Discount Factor ($\gamma$) | 0.999 |
| *Entropy ($\alpha$) Tuning* | |
| Initial Entropy ($\alpha_{\text{start}}$) | 1.0 |
| Minimum Entropy ($\alpha_{\min}$) | 0.2 |
| Entropy Decay Rate | 0.995 |
| *Prioritized Experience Replay (PER)* | |
| Buffer Size | 500,000 |
| Minimum Buffer Size to Train | 10,000 |
| Priority Exponent ($\alpha_{\text{buf}}$) | 1.0 |
| Initial Importance Sampling ($\beta$) | 0.2 |
| Maximum Importance Sampling ($\beta_{\max}$) | 0.9999 |

Table 3: Hyperparameters utilized for the Soft Actor-Critic (SAC) primary navigation agent.

# 5 Results

- **Performance Tables:** Table 4 details the mean cumulative reward and standard deviation achieved by the final hierarchical model across the three difficulty levels, evaluated both with and without wall obstacles.

| | **Cumulative Reward (Mean ± Std)** | |
|---|---|---|
| **Difficulty Level** | **Without Walls** | **With Walls** |
| Level 0 (Static) | $-2394.10 \pm 5372.75$ | $-51628.70 \pm 56995.14$ |
| Level 2 (Blinking) | $-4508.70 \pm 5791.70$ | $-70509.50 \pm 103038.03$ |
| Level 3 (Moving) | $-10808.60 \pm 13480.30$ | $-87513.70 \pm 87772.53$ |

Table 4: Mean cumulative reward and standard deviation evaluated across multiple difficulty modes and arena configurations.

- **Learning Curves & Metrics:**
  - *Loss and TD Error:* Both losses peaked around 100 episodes during initial aggressive exploration. Subsequently, critic loss and average TD error stabilized, indicating successful value mapping, followed by a smooth convergence of actor loss.
  - *Action Distribution and Q-Values:* By episode 80, behavior stabilized with the forward (FWD) action dominating at 90%. Q-values mirrored this, with FWD values distinctly surpassing rotational actions, validating the reward shaping.
  - *Task Performance:* Box attachment and boundary delivery success rates reached near 100% shortly after 100 episodes. Average episode length simultaneously dropped to 200 steps, reflecting optimized, direct-path navigation without unnecessary spinning.

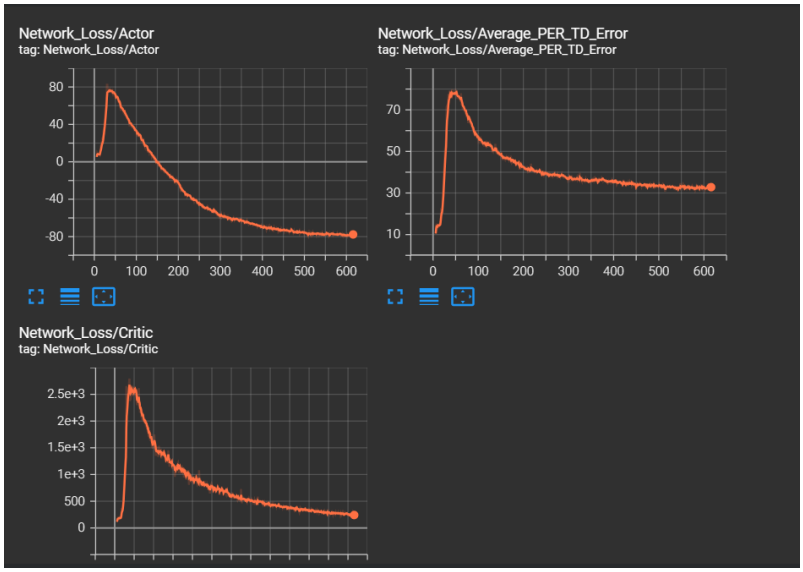



Figure 3: Actor loss, Critic loss and Avg TD error

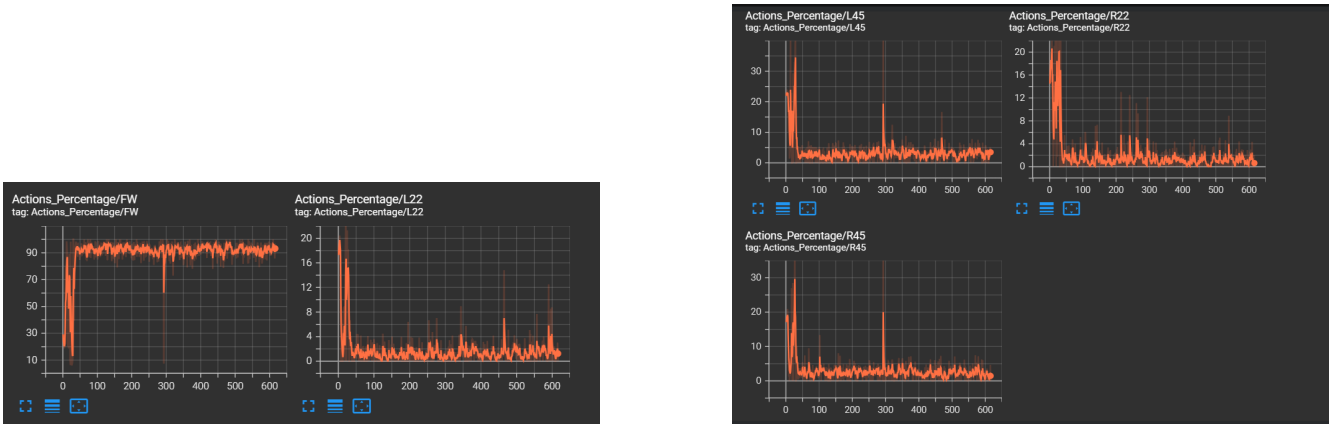



Figure 4: Action distribution percentage during training

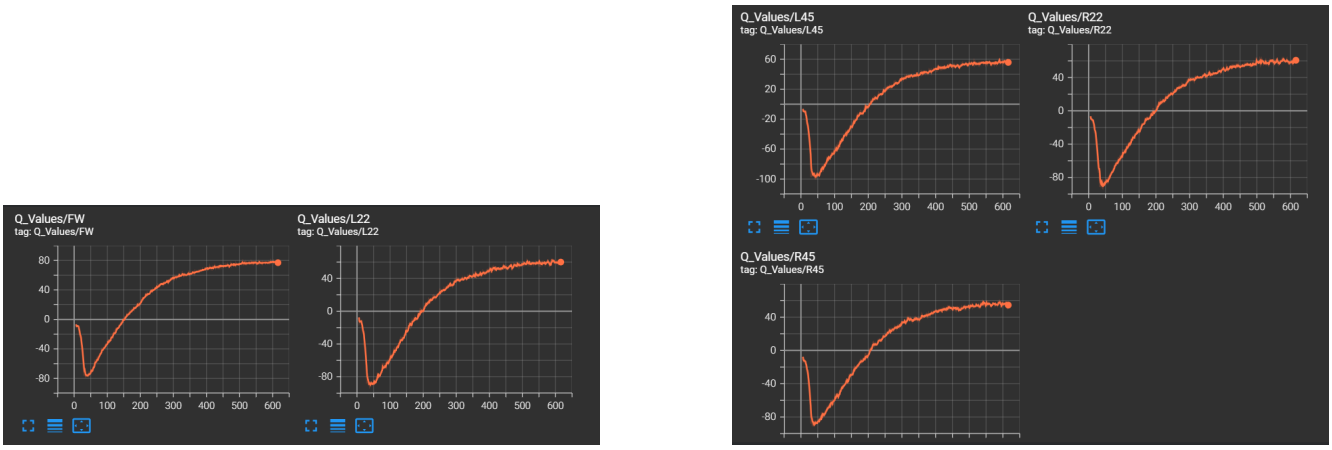
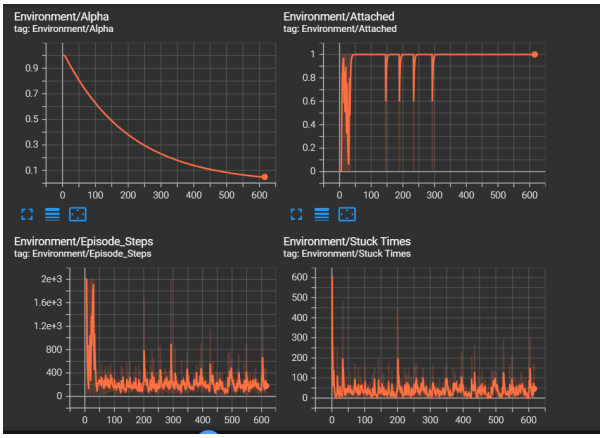
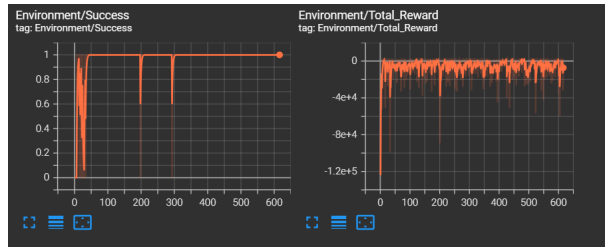



Figure 5: The Avg Q-values for all the actions during training




(a) the SAC entropy hyperparameter ($\alpha$), box attachment success rate, total steps taken per episode, and collision frequency (stuck occurrences)




(b) Success percentage and True reward

Figure 6: Training progression metrics

- **Overall Best Model:** The combined SAC + GRU hierarchical model proved vastly superior to standard Double DQN baselines. The functional isolation allowed the SAC network to specialize purely in target interception, while the Recurrent PPO prevented terminal failure states during extreme cornering.

# 6 Error Analysis and Discussion

- **Common Failure Modes:**
  - **Level 1 (Static):** Failures primarily stemmed from sensor aliasing. When placed in open space facing away from the box, the 18-bit sensor returns a null array, requiring the agent to rely purely on entropy for exploration, which occasionally caused timeouts.
  - **Level 2 (Blinking):** While frame stacking (4 frames) mitigated short blinks, extended periods of target invisibility caused the state representation to lose the box's signature, leading to catastrophic forgetting of the target location.
  - **Level 3 (Moving):** Intercepting a moving target requires estimating velocity vectors. The MLP-based SAC struggled to extrapolate complex, non-linear box trajectories purely from the concatenated 1D frame stack.
  - **With Walls:** Navigation failures occurred when the box was wedged deeply into a corner; the recovery GRU successfully freed the robot but often pushed the box further out of reach in the process.
- **Limitations & Future Improvements:** The primary limitation is the fixed size of the frame stack. With more time, replacing the SAC Actor's MLP with a transformer or LSTM could provide dynamic temporal receptive fields, solving the extended blinking issue.

# 7 Conclusion

- **Key Achievements:** I successfully demonstrated that complex POMDP robotic tasks can be broken down into specialized behavioral modules. Frame stacking combined with entropy-based SAC provided robust target acquisition, while the GRU-PPO integration drastically reduced terminal collision failures.
- **Most Important Takeaway:** In partially observable environments with severe consequences (getting stuck), a single end-to-end reactive policy is highly unstable. Hierarchical control mapping specialized networks to specific hardware states (the Stuck Bit) yields far better sample efficiency and robustness.
- **Future Directions:** Future work should focus on optimizing the agent's navigational efficiency, specifically by enhancing its pathfinding capabilities to form the shortest and most direct route to the target box in increasingly complex environments.

## Project Code and Video

The complete source code for the hierarchical SAC and Recurrent PPO architecture, including the custom environment wrappers, training loop, and hyperparameter configurations, is available on GitHub at: `https://github.com/Rishab2707/CS780-OBELIX`

A video demonstration showcasing the final trained agent can be viewed at: Evaluation Demo Video

## LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report? ChatGPT 5.x
- In which part did you use the LLM, and how did it help you?

| **Part** | **Used LLM?** | **Purpose of Using LLM** |
|---|---|---|
| Understanding concepts | Yes | Understand the principle behind each algorithm. |
| Ideation / brainstorming | No | |
| Debugging small code snippets | Yes | Debugging / Correcting some portions of code. |
| Plotting / visualization code | Yes | Generating plot of losses, neural network, etc. |
| Grammar / writing style | Yes | Corrected the grammar of small portions of text like figure captions. |
| Report structure / section ideas | No | |
| Other (specify) | None | None |

Table 5: Summary of LLM usage throughout the project.

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  Yes, below is the list of research papers I referred to during the project.
  - Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.* arXiv preprint arXiv:1406.1078.

- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.* International Conference on Machine Learning (ICML).
- Mahadevan, S., & Connell, J. (1992). *Automatic programming of behavior-based robots using reinforcement learning.* Artificial Intelligence, 55(2-3), 311-365.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). *Human-level control through deep reinforcement learning.* Nature, 518(7540), 529-533.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). *Prioritized Experience Replay.* International Conference on Learning Representations (ICLR).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms.* arXiv preprint arXiv:1707.06347.
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). *Dueling Network Architectures for Deep Reinforcement Learning.* International Conference on Machine Learning (ICML).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). *Human-level control through deep reinforcement learning.* Nature, 518(7540), 529-533.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.* arXiv preprint arXiv:1406.1078.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). *Prioritized Experience Replay.* International Conference on Learning Representations (ICLR).

- Apart from any LLM usage, what was your own contribution?
  I conceptualized and wrote the core codebase for the environment and the training pipeline along with selecting and implementing multiple RL algorithms, designing the hybrid architecture, executing the entire training and evaluation process, and systematically tuning the hyperparameters.

## References


[1] S. Mahadevan, and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," *Artificial Intelligence*, 55(2-3), 311–365, 1992.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.